Database II Lab/ 3rd Grade

[Second Semester] and [2026]

[Lab 9]

[28/4/2025] [8.30 + 10.30 am]

Instructor Information

Instructor	Email	Hours
Dr. Rasool Hisham Dr. Zainab Namh Dr. Azhar Flaih Assist. Prof. Zahraa Jaaz	[rasool.hisham@nahrainuniv.edu.iq]	[2 Hrs]

Creating RDLC Reports in C# with Visual Studio

Lab Objectives

By the end of this lab, students will be able to:

- Understand what RDLC reports are and when to use them.
- Explain WHY a DataSet is required for RDLC instead of a DataTable alone.
- Create a DataSet (.xsd) with a DataTable that defines the report's field structure.
- Design an RDLC report layout and bind fields from the DataSet.
- Write C# code to fill a DataTable with data from MySQL and display it in the report.

What is RDLC?

RDLC stands for **Report Definition Language Client-side**. It is a Microsoft reporting technology that allows you to create professional reports (tables, charts, headers, footers) inside .NET applications **without requiring a report server**. The report runs entirely on the client machine.

RDLC reports are typically used for printing structured data, generating invoices, employee lists, grade sheets, and similar documents.

The data flows through these components in this order:

Step	Flow
1	MySQL Database → Your C# code runs a SELECT query
2	C# code fills a DataTable (in memory) with the query results
3	You wrap the DataTable in a ReportDataSource and attach it to the ReportViewer
4	The ReportViewer reads the .rdlc layout file and renders the data on screen

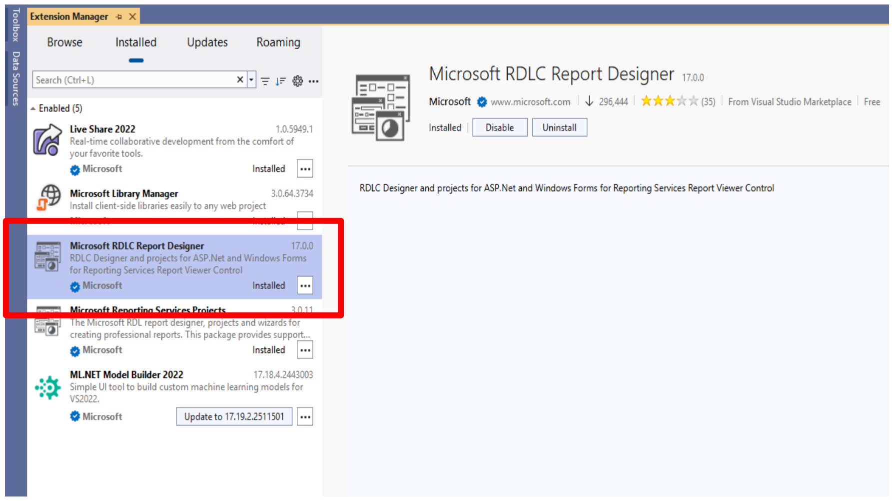
Create RDLC (Report Definition Language Client-side)

Step 1: Install the required Extensions

Microsoft RDLC Report Designer Extension enables **graphical design of RDLC reports** (*.rdlc files) directly inside Visual Studio. It brings in a **drag-and-drop interface** for creating professional-looking reports—tables, charts, headers, footers, data bindings, and more.

It is just like "Microsoft Word for Reports" inside Visual Studio.

Extensions → Manage Extensions so that the Add report (*.rdlc) appears

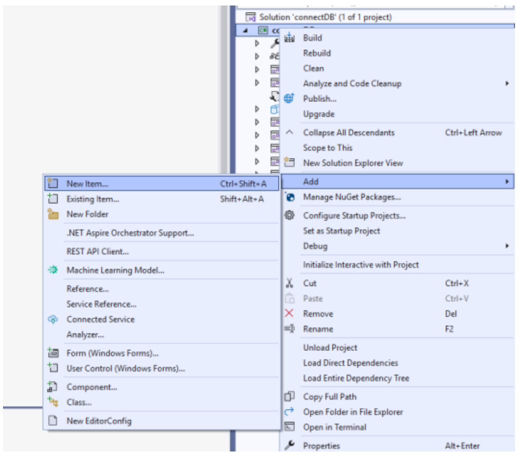


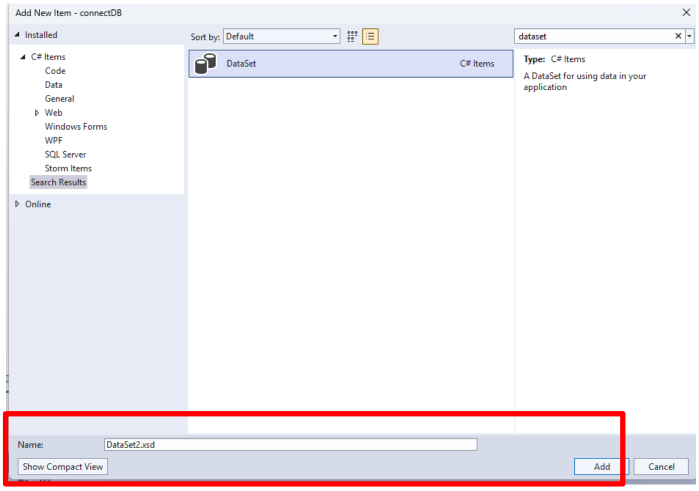
Step 2: Create a Dataset

1. **Add a New Dataset**:
   - Right-click on the **Project** name in the **Solution Explorer**.
   - Select **Add** → **New Item** → **Dataset**.
   - A dataset will be created that serves as the structure for your data.
2. **Add Data Table**:
   - Inside the Dataset, R-click inside it and add a **new DataTable**.
   - Name the DataTable (e.g. ContactsTable).
   - Right-click the DataTable and define the required **columns** (e.g., ContactName,ContactTel, …).
   - Save and close the dataset.

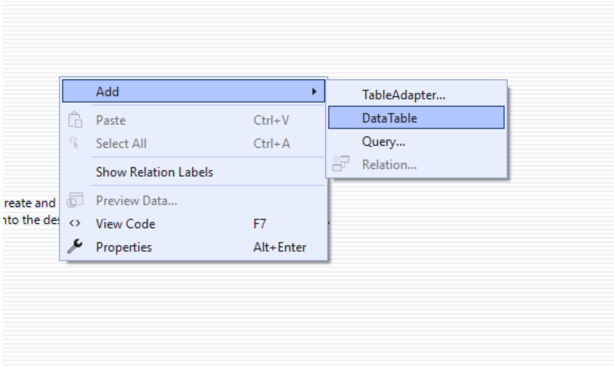
Why do we need this? The dataset acts as a ***container for data*** that you will display in the RDLC report. It is required to ***structure the data*** you intend to visualize and to provide it to the report.

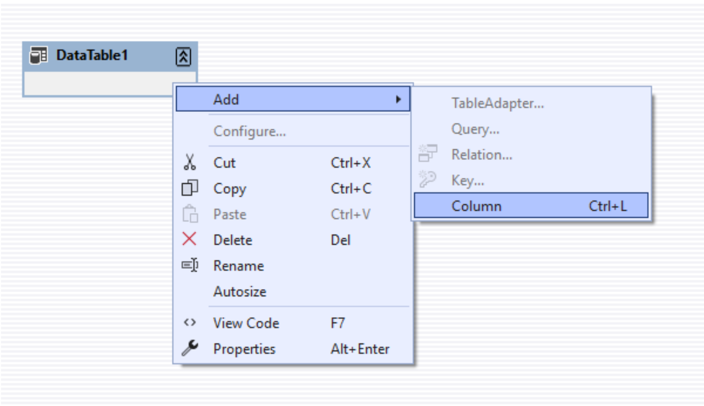
> **Why these specific names?**
> These column names MUST match the column names (or aliases) returned by your SQL query at runtime.



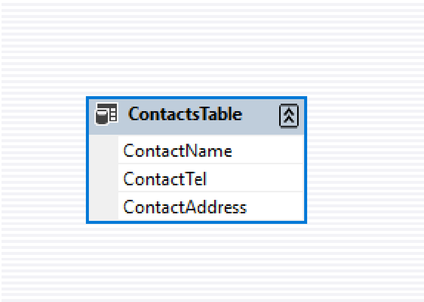


In this window, include the necessary columns for the report.





This is the final report fields:



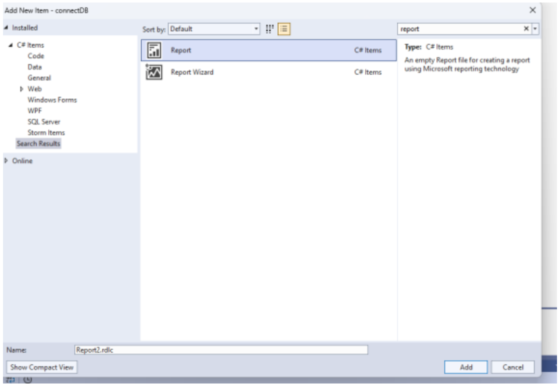
Step 3: Create a New Report

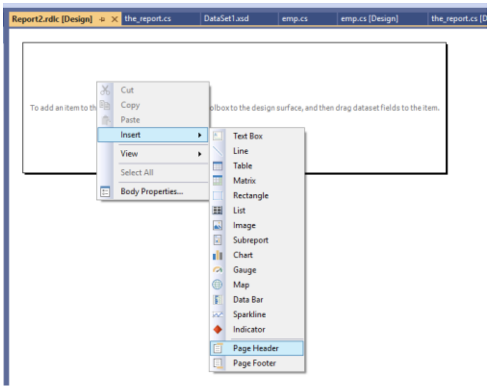
1. **Add a Report**:
   - Right-click on the **Project** name again.
   - Select **Add** → **New Item** → **Report**.
   - This will add a new *.rdlc file to your project.

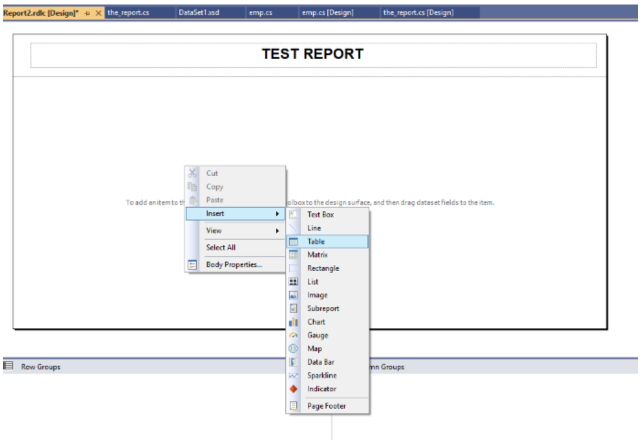
2. **Design the Report**:

- In the RDLC file, go to the **Design View**.
- Add **fields** to the **Body** section to represent your data (e.g., ContactName, …).
- Add a **Textbox** in the **Header** to show the title or other static information.

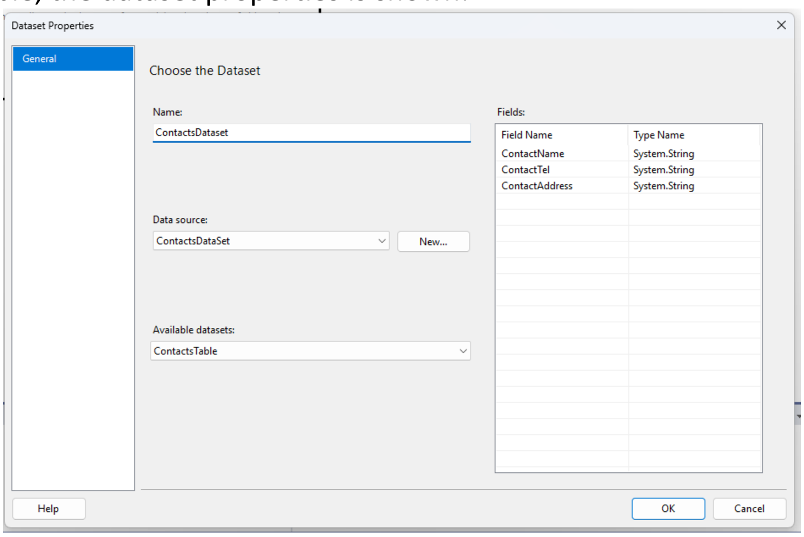
Why do we need this? The report file (RDLC) contains the ***layout and structure of the report***, defining where the data will be displayed, how it will appear, and the layout of tables, charts, or any other visual elements.







When adding the table, the dataset properties is shown:

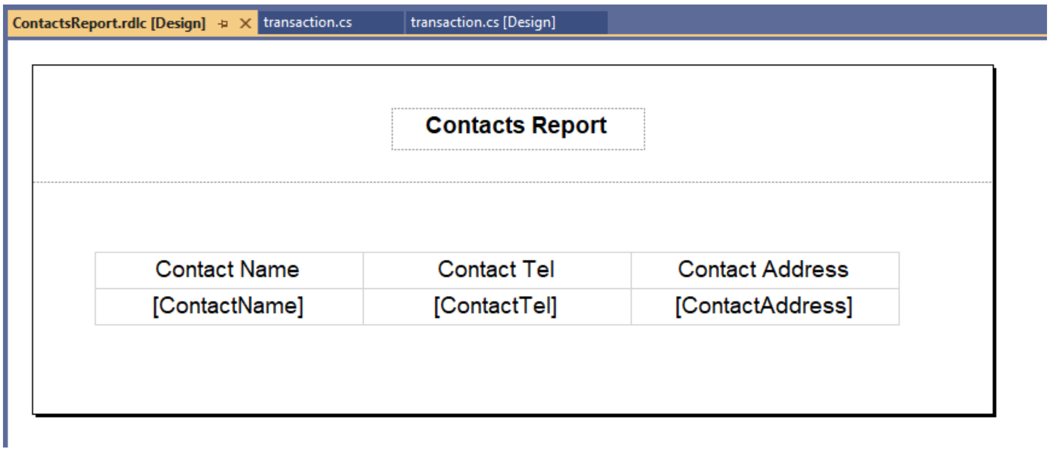


Define the dataset name, data source and the datatable:

Dataset Name is the internal name for use within the report. C# code use this name as the source of the report.

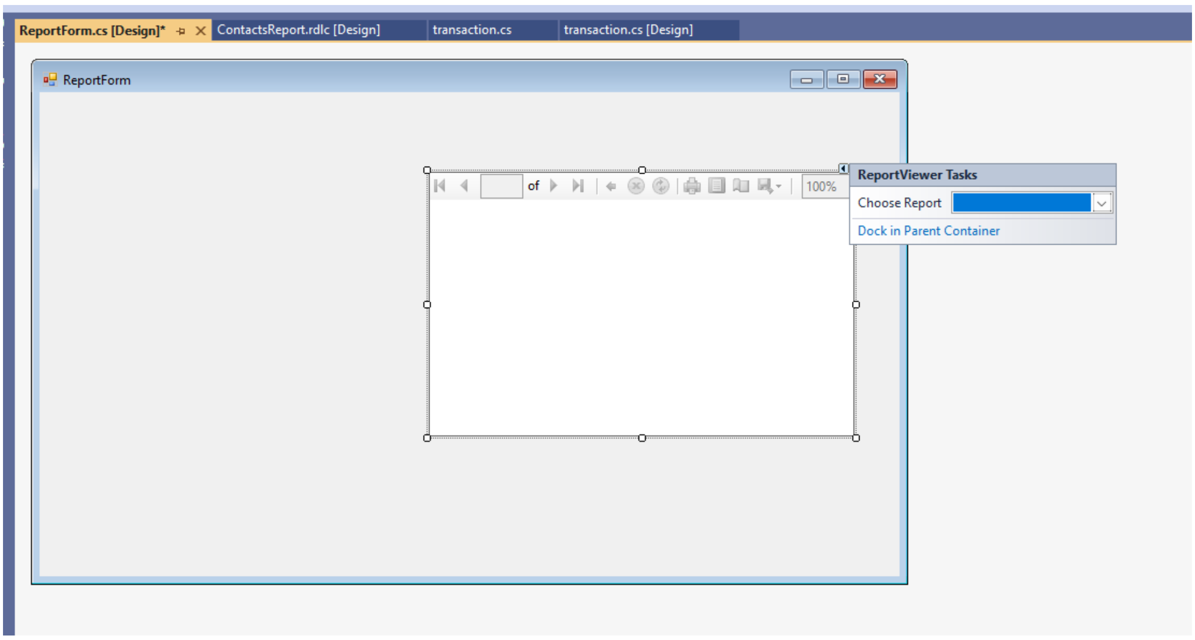
Dataset Source is your (.xsd file). The DataSet created in the previous step.

Then add the required fields to the report table:

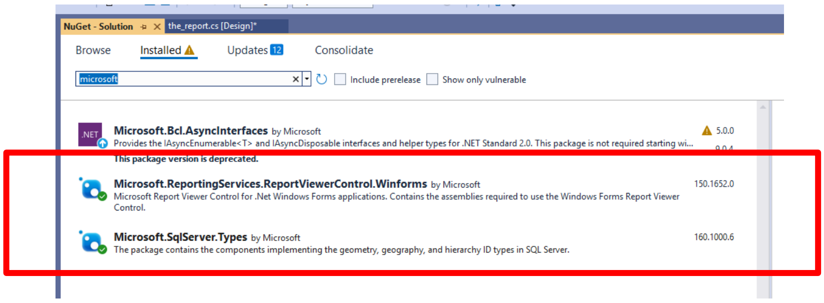


Step 4: Add the ReportViewer Control

1. **Add a ReportViewer**:
   - Open your Windows Forms application.
   - Drag the **ReportViewer** control from the **Toolbox** onto your form.



   - If the ReportViewer control is not available in the toolbox, you may need to install it using the **Manage NuGet Packages** feature. Search for **Microsoft.ReportingServices.ReportViewerControl.Winforms** and install



2. **Set Up ReportViewer**:

   - Inside the Form code (Form.cs), at the top add the necessary **namespace** for RDLC:

```
      using Microsoft.Reporting.WinForms;
```

Why do we need this? The ReportViewer control is used to display the RDLC report within your Windows Forms application. It provides an interactive viewer for rendering reports.

Step 5: Bind Data to the Report (C# Code)

```
    private void RepView_Load(object sender, EventArgs e)
    {
        // STEP A: Clear any old data sources
        RepView.LocalReport.DataSources.Clear();

        // STEP B: Set the report path (use relative path!)
        RepView.LocalReport.ReportPath = "ContactsReport.rdlc";

        // STEP C: Fill a DataTable from MySQL
        DataTable dt = new DataTable();
        using (var conn = new MySqlConnection(connectionString))
        {
            conn.Open();
            // Column names MUST match the DataSet (.xsd) columns!
            string sql = "SELECT ContactName,
ContactTel,ContactAddress from contacts”;
            using (MySqlCommand cmd = new MySqlCommand(sql, conn))
             {
               var adapter = new MySqlDataAdapter(cmd);
               adapter.Fill(dt);
             }
        }
        // STEP D: Create a ReportDataSource
        // "ContactsDataset" MUST match the Dataset Name inside the
.rdlc report
        ReportDataSource rds = new
ReportDataSource("ContactsDataset", dt);

        // STEP E: Attach the data source to the report
        RepView.LocalReport.DataSources.Add(rds);

        // STEP F: Render the report
        RepView.RefreshReport();
    }}
```

Line-by-Line Explanation

Line	What It Does	Why It’s Needed
DataSources.Clear()	Removes any previously attached data sources.	This line clears any existing data sources that are already added to the LocalReport
ReportPath = "..."	Tells the ReportViewer WHERE the .rdlc layout file is.	Without this, the viewer has no layout to render — it doesn’t know where to put the data.
DataTable dt + adapter.Fill(dt)	Runs the SQL query and stores the results in a DataTable in memory.	This is the ACTUAL DATA. The DataSet (.xsd) was just the design.
new ReportDataSource("ContactsDataset", dt)	Used to bind data to the report. The constructor takes two parameters: DataSet and DataTable.	" ContactsDataset ": This is the name of the **dataset** or data source defined in the **RDLC** report. It must match the name defined in the report design. dt: This is the **DataTable** (dt), which contains the actual data that will be displayed in the report.
DataSources.Add(rds)	Attaches the data source to the report.	The report now has both a layout (.rdlc) and data (DataTable). It can render.
RefreshReport()	Triggers the ReportViewer control to refresh and render the report.	Without this, the viewer does not update. The data is attached but not displayed.

Set "Copy to Output Directory" to "Copy always" (as shown below) so the .rdlc file is automatically copied to the bin\Debug folder whenever you build — this way ReportPath = "ContactsReport.rdlc" can find the file using a simple relative path that works on any computer, not just yours.